

Pacman-Arena

Lasst eure selbstgebauten Pacman-Bots gegeneinander antreten — auf genau der Spiel-Engine, die im Unterricht benutzt wird. Am Ende steht eine Tabelle, wer gewonnen hat, und eine Auswertung, **warum**.

Inhalt

1. Was ihr braucht
2. Schritt 1: Entpacken
3. Schritt 2: PowerShell im richtigen Ordner öffnen
4. Schritt 3: Der erste Lauf
5. Schritt 4: Euren eigenen Bot einbauen
6. Alle Befehle
7. Die Ausgabe lesen
8. Wie viele Partien braucht man?
9. Wenn etwas nicht klappt
10. Alle Optionen auf einen Blick
11. Über die Füllspieler
12. Ein Hinweis zur Sicherheit
13. Was noch im Ordner liegt

Was ihr braucht

Python 3.8 oder neuer. Sonst nichts. Kein `pip install`, keine Anmeldung, kein Internet.

Schritt 1: Entpacken

Entpackt die ZIP-Datei irgendwohin, wo ihr sie wiederfindet. Ihr bekommt einen Ordner namens `PacmanArena`. Zum Beispiel:

```
H:\Informatik\Pacman\PacmanArena
```

Merkt euch diesen Pfad — den braucht ihr gleich.

Wichtig: Manche Entpack-Programme legen einen Ordner *im* Ordner an, also `PacmanArena\PacmanArena`. Öffnet den entpackten Ordner und schaut nach: Ihr müsst darin `README.md`, `Pacman.py` und einen Ordner `arena` sehen. Seht ihr stattdessen wieder nur einen Ordner `PacmanArena`, dann ist *der* der richtige.

Schritt 2: PowerShell im richtigen Ordner öffnen

Es gibt zwei Wege. Der erste ist schneller, der zweite funktioniert immer.

Weg A — direkt aus dem Explorer

Öffnet den Ordner `PacmanArena` im Explorer. Klickt oben in die **Adressleiste**, sodass der Pfad blau markiert ist. Tippt `powershell` und drückt Enter.

Es öffnet sich ein blaues Fenster, das schon im richtigen Ordner steht. Fertig.

Weg B — mit `cd` hinnavigieren

`cd` heißt „change directory“, also Ordner wechseln. Startmenü → `powershell` eingeben → Enter. Dann:

```
cd H:\Informatik\Pacman\PacmanArena
```

Also `cd`, ein Leerzeichen, und der Pfad zu eurem Ordner.

Tipp, der euch das Tippen spart: Schreibt `cd` (mit Leerzeichen), zieht dann den Ordner `PacmanArena` aus dem Explorer in das PowerShell-Fenster und drückt Enter. Der Pfad wird automatisch eingefügt.

Wenn der Ordner auf einem anderen Laufwerk liegt — etwa auf `H:`, während PowerShell in `C:` startet — müsst ihr erst das Laufwerk wechseln:

```
H:
cd H:\Informatik\Pacman\PacmanArena
```

Die vier Befehle zum Navigieren

BEFEHL	WAS ER TUT
<code>dir</code>	zeigt, was im aktuellen Ordner liegt
<code>pwd</code>	zeigt, in welchem Ordner ihr gerade steht
<code>cd unterordner</code>	geht hinein in einen Unterordner
<code>cd ..</code>	geht einen Ordner zurück (nach oben)

`cd ..` braucht ihr, wenn ihr euch verlaufen habt oder aus Versehen einen Ordner zu tief gelandet seid. Zweimal zurück geht mit `cd ../../`.

Prüfen, ob ihr richtig steht

```
dir
```

Ihr müsst **arena**, **Pacman.py** und **README.md** in der Liste sehen. Seht ihr etwas anderes, seid ihr im falschen Ordner.

Zur Sicherheit noch dieser Befehl:

```
python arena/freundschaftsarena.py --help
```

Kommt eine Liste von Optionen, passt alles. Kommt **No such file or directory**, steht ihr im falschen Ordner — zurück zu Weg A.

Kommt **python: Der Begriff "python" wurde nicht erkannt**, probiert es mit **py** statt **python**. In WinPython heißt der Startbefehl manchmal auch anders; dann benutzt ihr überall in diesem README **py** statt **python**.

Schritt 3: Der erste Lauf

```
python arena/freundschaftsarena.py --partien 20 --fueller 4 --grafisch
```

Das spielt 20 Partien und öffnet danach eine Seite im Browser:

- oben die **Tabelle** über alle 20 Partien
- darunter **eine Partie zum Abspielen**, Zug für Zug, mit Schieberegler
- darunter der **Stärkeverlauf** und eine Karte je Bot

Es entsteht dabei die Datei `arena_turnier.html` in eurem Ordner. Die könnt ihr verschicken — sie funktioniert überall, auch auf Rechnern ohne Python.

Öffnet der Browser sich nicht von selbst:

```
start arena_turnier.html
```

Schritt 4: Euren eigenen Bot einbauen

Das ist alles: Datei in den Ordner `arena\bots\` legen. Nichts anmelden, nichts importieren, nichts eintragen. Beim nächsten Start ist euer Bot dabei.

Konkret:

1. Öffnet im Explorer den Ordner `PacmanArena\arena\bots`.
2. Kopiert eure `.py`-Datei hinein — einfach per Drag & Drop.
3. Startet die Arena wie in Schritt 3.
4. Ganz oben in der Ausgabe steht `gefunden: EuerBotName`.

Steht euer Bot nicht in dieser Liste, stimmt etwas mit der Datei nicht — siehe „Wenn etwas nicht klappt“ weiter unten.

Zwei Dateien liegen schon drin:

DATEI

<code>beispiel_gerader_fresser.py</code>	Kopiervorlage, einfache Strategie
<code>ClaudeEndboss.py</code>	ein sehr starker Bot, als Maßstab

Am schnellsten geht es so: `beispiel_gerader_fresser.py` kopieren, umbenennen (z. B. `mein_bot.py`), die Klasse darin umbenennen, und dann Stück für Stück ändern.

Das kleinstmögliche Beispiel

```
from Pacman import Pacman, Direction

class MeinBot(Pacman):
    def TurnOrMoveOrStill(self):
        self._Move()          # läuft immer geradeaus
```

Speichern als `arena\bots\mein_bot.py`, Arena starten — er ist dabei.

Ein eigenes Bild für euren Bot

Im Fenster (--fenster, und auch in `PacmanGame.py`) wird jeder Bot als Bild gezeichnet. Welches, steht in einer Zeile im Konstruktor:

```
self.icon = "icons/MeinBot.png"
```

Legt das Bild als PNG nach `icons/`. Drei Dinge gibt der Renderer vor:

Das Bild muss nach rechts schauen. Osten ist die ungedrehte Grundstellung; alle anderen Richtungen dreht der Renderer daraus. Ein Bild, das nach oben schaut, läuft im Spiel seitwärts.

Es wird starr gedreht, nicht gespiegelt. Schaut euer Bot nach Westen, steht das Bild auf dem Kopf. Alles mit einem klaren Oben und Unten — eine Krone, ein einzelnes Auge — sieht dann falsch aus. Zeichnet die Figur deshalb **symmetrisch zur Waagerechten**, und lasst nur das Maul die Richtung anzeigen. Der mitgelieferte `ClaudeEndboss` macht genau das.

Am Ende sind es 32 Pixel. Der Renderer skaliert jedes Bild auf Zellengröße. Feine Linien verschwinden; was zählt, ist die Silhouette. Die Bilder des Lehrers sind 40×40 , das passt gut.

Findet der Renderer die Datei nicht, stürzt nichts ab — er malt stattdessen den farbigen Kreis. Ein Tippfehler im Pfad fällt also nur daran auf, dass **kein** Bild erscheint.

Wer sich eins erzeugen lassen will: `python scripts/mach_icon.py icons/MeinBot.png` zeichnet die Vorlage, aus der `ClaudeEndboss.png` entstanden ist — Farben und Form stehen oben in der Datei.

Die drei Regeln der Engine

1. Eure Klasse muss von `Pacman` erben.
2. Ihr überschreibt `TurnOrMoveOrStill`. Pro Zug genau **eine** Sache: * `self.direction = Direction.north` — drehen * `self._Move()` — einen Schritt in Blickrichtung * gar nichts — stehenbleiben
3. Der Rückgabewert wird ignoriert.

Drehen kostet einen ganzen Zug. Das ist die wichtigste Regel im Spiel: Wer sich viel dreht, frisst wenig.

Was ihr abfragen könnt

```
self.position._x, self.position._y    # wo ihr steht
self.direction                        # wohin ihr schaut
self.strength                         # eure Stärke
self._field[Position(x, y)]           # was auf einem Feld liegt
Position.fieldsize                    # Kantenlänge des Bretts
```

Was auf einem Feld liegen kann: `Cabbage` (Kohl, macht stärker), `Empty`, `Wall`, oder ein `Pacman` (ein Gegner). Prüfen mit `isinstance`:

```
from Pacman import Cabbage, Position
ziel = self._field[Position(x, y)]
if isinstance(ziel, Cabbage):
    ...
```

Das Brett ist ein Torus — wer rechts hinausläuft, kommt links wieder herein. Rechnet Koordinaten deshalb immer mit `% Position.fieldsize`.

Alle Befehle

Turnier spielen

```
python arena/freundschaftsarena.py --partien 20 --fueller 4
```

Tabelle im Terminal, danach eine Diagnose je Bot.

```
python arena/freundschaftsarena.py --partien 20 --fueller 4 --grafisch
```

Dasselbe, zusätzlich als Seite im Browser.

```
python arena/freundschaftsarena.py --partien 200 --fueller 14
```

Auf Turniergröße. Wenn in der Klasse rund 15 Bots antreten, sagt eine Sechser-Runde wenig: Auf dem Brett kommen dann statt 32 nur noch 13 Kohl auf jeden Spieler. Wer auf lange Fressbahnen gebaut hat, findet keine mehr.

Zuschauen

```
python arena/freundschaftsarena.py --partien 1 --replay partie.html
```

Schreibt eine HTML-Datei zum Abspielen — ohne den Browser zu öffnen.

```
python arena/freundschaftsarena.py --replay-saat 7 --grafisch
```

Zeigt unten gezielt Partie Nummer 7 statt der ersten. Nützlich, wenn ihr sehen wollt, wie genau euer Bot in *der* *einen* Partie verloren hat.

```
python arena/freundschaftsarena.py --fenster --fueller 4 --fps 6
```

Live im Fenster, wie `PacmanGame.py`. Leertaste hält an, `+` und `-` ändern das Tempo, Esc schließt.

Braucht `pygame` — falls es fehlt: `pip install pygame`.

Herausfinden, warum ein Bot verloren hat

```
python arena/freundschaftsarena.py --partien 1 --warum bericht.md
```

Schreibt einen kurzen Bericht (~7 KB): zu **jedem Todesfall** die acht Züge davor, mit der Lage, die jeweils vorlag. So sieht eine Zeile aus:

```
Zug 60 Fueller1 11,2 Blick S k=37 Bahn N0 S0 W0 00
-> gedreht/in Zug 61 gefressen von ClaudeEndboss
| naechster ClaudeEndboss d=1 k=40 Blick S ich 0.90 er 0.92
```

Bahn N4 heißt vier Kohl am Stück nach Norden. **ich** und **er** sind die **echten Kampfchancen** der Engine. Oben schauen beide nach Süden — also steht der Gegner im Rücken und gewinnt mit 92 %.

Der Bericht ist klein genug, um ihn samt eurem Bot-Quelltext einer KI vorzulegen und zu fragen, was der Bot hätte tun sollen.

```
python arena/freundschaftsarena.py --partien 1 --protokoll zuege.jsonl
```

Dasselbe vollständig, eine Zeile je Zug je Bot, zum Auswerten mit eigenem Code.

Nachprüfen, dass die Arena richtig zählt

```
python -m unittest arena.tests.test_arena
```

16 Tests. Müssen alle grün sein.

Die Ausgabe lesen

Spieler	staerkster	allein uebrig	lebt	Staerke	Leerlauf	ms/Zug
ClaudeEndboss	50.0% [30%,70%]	30.0%	60%	90.0	0%	6.40

SPALTE	BEDEUTUNG
stärkster	Anteil der Partien, in denen der Bot am Ende die höchste Stärke hatte
[30%,70%]	Konfidenzintervall — dazu unten mehr, es ist wichtiger als die Zahl davor
allein übrig	Anteil der Partien, die er als Letzter überlebt hat. Das ist die eigentliche Siegbedingung
lebt	Anteil, in dem er das Ende erlebt hat
Stärke	Durchschnitt am Partieende
Leerlauf	Züge ohne jede Wirkung — die wichtigste Zahl für die Fehlersuche
ms/Zug	Rechenzeit je Entscheidung

Die drei Zahlen, die Fehler verraten

Leerlauf über 15 % — Züge, in denen sich weder Position noch Blickrichtung geändert haben: stehengeblieben, gegen eine Wand gelaufen, oder in die Richtung gedreht, in die der Bot schon schaute. Jeder davon ist ein geschenkter Zug. Der Beispiel-Bot stand in seiner ersten Fassung bei **94 %** — er lief bis zum

Partieende gegen eine Wand, weil `_Move()` gegen eine Wand einfach nichts tut. Ohne diese Spalte fällt so etwas nie auf.

Angriffe gewonnen unter 50 % — der Bot greift zu oft von vorne an. Die Engine rechnet $a / (a + b)$, wobei die Stärke des Verteidigers geteilt wird:

ANGRIFF	TEILER	EURE CHANCE BEI GLEICHER STÄRKE
von hinten	10	91 %
von der Seite	5	83 %
von vorne	1	50 %

davon aus Kämpfen bei 0 % — die zweite Spielhälfte hat keinen Kohl mehr. Ein Bot, der dann nicht kämpft, hört ab der Mitte auf zu wachsen.

Und einer, den man leicht übersieht: **verteidigt**. Wer angegriffen wird und gewinnt, erbt die **volle** Stärke des Angreifers. Das ist der schnellste Weg nach oben — und man kann ihn nicht planen, nur wahrscheinlicher machen, indem man Stärkeren nicht den Rücken zudreht.

Wie viele Partien braucht man?

Die unbequemste Frage, und die wichtigste.

PARTIEN	GENAUIGKEIT EINER QUOTE
20	±21 Punkte
100	±9 Punkte
400	±5 Punkte
2000	±2 Punkte

Bei 20 Partien kann ein Bot mit 40 % echter Stärke leicht als 60 % erscheinen. **Wer nach 20 Partien einen Sieger ausruft, misst Rauschen.**

Für „ist meine Änderung besser?“ fangt bei **200 Partien** an — und lässt beide Fassungen mit derselben `--saat` laufen, damit sie dieselben Bretter sehen. Das allein macht den Vergleich rund viermal so aussagekräftig, weil die Unterschiede zwischen den Brettern herausfallen.

Wenn etwas nicht klappt

unrecognized arguments: --grafisch Ihr habt eine ältere Fassung der Arena. Ersetzt den ganzen `PacmanArena`-Ordner durch den neuen. Bis dahin tut es `--replay partie.html`.

No such file or directory: arena/freundschaftsarena.py PowerShell steht im falschen Ordner. Tippt `dir` — ihr müsst `arena`, `Pacman.py` und `README.md` sehen. Wenn nicht: `cd ..` geht einen Ordner zurück, dann noch mal schauen. Am einfachsten ist Weg A aus Schritt 2.

python: Der Begriff "python" wurde nicht erkannt Probiert `py` statt `python`. Falls auch das nichts bringt, ist Python nicht im Suchpfad — dann startet PowerShell über die WinPython-Verknüpfung statt über das Startmenü.

ModuleNotFoundError: No module named 'pygame' Nur `--fenster` braucht `pygame`. Entweder `pip install pygame`, oder statt dessen `--grafisch` benutzen — das braucht nichts.

Mein Bot taucht nicht in der Tabelle auf Die Arena meldet beim Start, was sie gefunden hat („gefunden: MeinBot“). Fehlt er, geht diese Liste durch:

- Liegt die Datei wirklich in `arena\\bots\\` ? (Nicht daneben, nicht eine Ebene höher.)
- Endet sie auf `.py` ? Windows blendet Endungen oft aus — im Explorer unter *Ansicht* → *Dateinamenerweiterungen* einschalten und nachsehen, dass die Datei nicht `mein_bot.py.txt` heißt.
- Beginnt der Dateiname mit `_` ? Solche werden absichtlich übersprungen.
- Erbt eure Klasse von `Pacman`, also `class MeinBot(Pacman):` ?

!! meinbot.py laesst sich nicht laden: ... Ein Fehler in eurer Datei. Die Meldung dahinter sagt welcher. Das Turnier läuft ohne euch weiter — eine kaputte Datei soll nicht alle anderen blockieren.

Der Bot stürzt mitten im Spiel ab Dann steht `FEHLER` hinter ihm in der Tabelle. Die Arena fängt das ab und zählt den Zug als verloren, damit die anderen weiterspielen können.

Alle Optionen auf einen Blick

OPTION	STANDARD	
<code>--partien N</code>	20	wie viele Partien
<code>--fueller N</code>	0	zusätzliche Gegner, die nicht würfeln
<code>--saat N</code>	1	Zufallsstart; gleiche Saat = exakt gleiche Partien
<code>--grenze N</code>	1500	Zuglimit (die Engine selbst hat keines)
<code>--feldgroesse N</code>	15	Kantenlänge des Bretts
<code>--ohne-waende</code>		ohne die sechs Wände aus <code>PacmanGame.py</code>
<code>--ordner PFAD</code>	<code>arena/bots</code>	wo die Bot-Dateien liegen
<code>--grafisch</code>		Turnier + Replay als HTML, öffnet den Browser
<code>--replay DATEI</code>		dasselbe, ohne den Browser zu öffnen
<code>--replay-saat N</code>	<code>--saat</code>	welche Partie unten gezeigt wird
<code>--fenster</code>		live im pygame-Fenster
<code>--fps N</code>	12	Tempo im Fenster
<code>--warum DATEI</code>		Bericht zu jedem Todesfall (Markdown)
<code>--protokoll DATEI</code>		jeder Zug jedes Bots (JSONL)
<code>--bericht DATEI</code>		alle Partien als Rohdaten (JSON)
<code>--nur-tabelle</code>		ohne die ausführliche Diagnose

Jederzeit abrufbar mit:

```
python arena/freundschaftsarena.py --help
```

Über die Füllspieler

`--fueller N` erzeugt N Gegner, die **nicht würfeln**. Das ist Absicht: Der Zufallsbot der Engine trifft in der Hälfte seiner Züge gar keine Entscheidung, und gegen den zu gewinnen sagt fast nichts über die eigene Strategie.

Die Füller hier fressen, was vor ihnen liegt, weichen Stärkeren aus und laufen nicht gegen Wände. Jeder bekommt eine andere „Stufe“ zwischen 0 und 1, die steuert, wie oft er sich die längste Kohlbahn sucht statt einfach weiterzulaufen — so besteht das Feld nicht aus einer Meinung in fünffacher Ausfertigung.

Ein Hinweis zur Sicherheit

Die Dateien in `arena/bots/` werden ganz normal ausgeführt, mit allen Rechten, die das Skript hat. Legt nur Dateien hinein, deren Herkunft ihr kennt.

Was noch im Ordner liegt

<code>arena/ANLEITUNG.md</code>	die ausführliche Fassung, mit dem Format des Zugprotokolls
<code>arena/freundschaftsarena.py</code>	die Arena selbst
<code>arena/protokoll.py</code>	das Zugprotokoll und der Bericht
<code>arena/tests/</code>	16 Tests, die die Buchhaltung prüfen
<code>Pacman.py</code> , <code>TRex.py</code> , <code>PacmanRenderer.py</code>	die Engine, unverändert
<code>icons/</code>	Bilder für die Fensteransicht